Link-Layer Services and Framing

Lecture 25

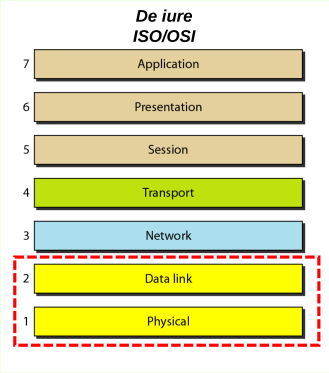
Computer Networks and Distributed Programming

What one hop actually is

- Every path we computed is a sequence of single hops, and we never asked what one of them is.
- Below the network layer there is no more routing to do: there are two machines joined by one channel, and bits to get across.
- Two questions were left open: how the bits are packed on that channel, and how the machine at the other end is named there.
- Today the first: what a link layer is asked to do, and how a receiver finds where one frame ends and the next begins.
- The name comes with local networks, where more than two machines share the same wire and one has to be addressed.

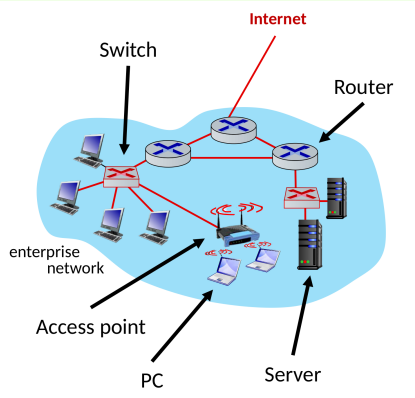
From the network layer down

- The network layer moves a datagram between two hosts wherever they are, and it does it by routing.
- The two layers below concern two machines already connected: one stretch of the path, no choices to make.
- Link: the transmission of packets over one link, from a node to the next one.
- Physical: what the link is made of — medium, connectors, cables — and how a bit is represented on it.
- Two routers joined by a line: everything this layer does happens on that segment.



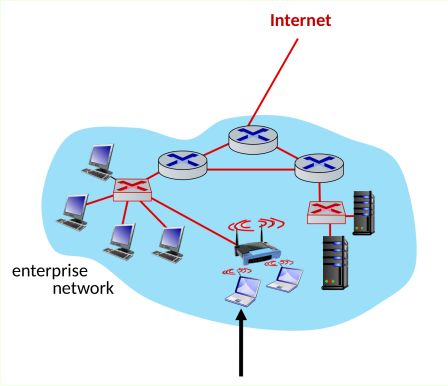
Nodes and links

- Node: any device that runs a link-layer protocol: hosts, routers, switches, hubs, wireless access points.
- Link: the communication channel that connects two adjacent nodes along a path.
- Over a link, the transmitting node encapsulates the datagram in a frame and puts the frame on the channel.
- In the graph we routed on, an edge was a wire between two routers. A link need not be that.
- It can be a medium shared by many nodes at once — and most of what this layer does exists because of it.



Six links between a laptop and a server

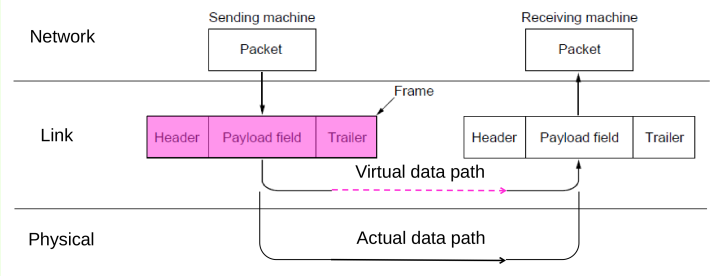
- Follow a datagram from the wireless host to the server at the other end of this network.
- WiFi to the access point, Ethernet to the switch, switch to router, router to router, then down to the other switch and the server: six links.
- Every one of them carries the datagram inside a frame of its own, built and thrown away on that hop.
- Four link technologies, four frame formats, one datagram unchanged from end to end.



A different protocol on every hop

- The links along a path are not alike: radio here, twisted pair there, fibre in the middle, and each has its own protocol.
- At every node the frame is opened, the datagram is taken out, and it is packed again into a frame of a different kind.
- The services differ too. One link may acknowledge every frame; the next one may drop frames and say nothing.
- So nothing above can rely on them: IP is designed to run over whatever happens to be there, which is why the guarantees are rebuilt end to end.
- What crosses unchanged is the datagram — give or take the two fields a router rewrites — and what changes at every step is the wrapping.

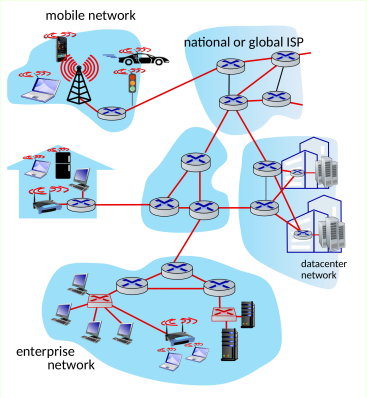
The last encapsulation



- Header, payload, trailer: the first time a layer puts something *after* the data, because the check covers all of it.
- The largest payload a frame will carry is the MTU, the number that decided where a datagram had to be cut.
- It is convenient to think of two link layers talking. The bits actually go down, across, and up again.

Two layers that are in every device

- Application and transport live only in the end systems: a switch has no transport layer, and a router has no application.
- Link and physical are in every piece of the infrastructure — hosts, routers, switches, hubs, access points — and on every link, wired or wireless.
- They are also tightly intertwined, so much so that they are often grouped into one layer called *network access*.
- Ethernet is the plain case: one name for a set of standards that specify both the frame and the signal on the wire.



What a link layer may offer

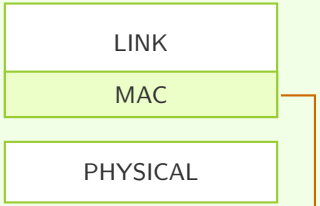
Four services

A link-layer protocol may offer up to four things. Which ones it offers is a design choice, and different links choose differently.

- Framing: put the datagram inside a frame with a header and a trailer, whose structure the protocol fixes.
- Link access: the rules for putting a frame on the medium, laid down by a medium access control protocol.
- Reliable delivery: guarantee that a frame handed to this link arrives at the other end of it.
- Error detection and correction: bits computed by the sender, checked — and sometimes used to repair — by the receiver.
- Only the first is offered by essentially every protocol. The other three are optional, and the reasons are worth the time.

Link access and the MAC sublayer

- Point-to-point: one sender, one receiver. The access rules are simple or absent: send whenever the link is idle.
- Broadcast: many nodes on one medium, and everything sent is heard by all of them. Two that transmit together destroy each other's frame.
- That is the multiple access problem, and the MAC protocol exists to coordinate who transmits when.
- Careful with the word: broadcast here is the nature of the channel, not sending a message to everybody.



Medium Access Control: the sublayer that decides when we may transmit

The rules that arbitrate a shared medium are the next thing we look at.

Reliable delivery on one link

- The three guarantees are the ones we already took apart: nothing lost, nothing duplicated, everything in order — rebuilt here one hop at a time.
- The machinery is the same: acknowledgements, timers, sequence numbers, retransmission. Only the distance is different.
- Where it pays: on links that lose often, such as wireless: the error is repaired where it happened, in a fraction of a round trip.
- Where it does not: on fibre, coax and twisted pair the error rate is so low that the acknowledgements cost more than the losses.
- Which is why many wired protocols — Ethernet among them — simply do not offer it.

Three classes of service

Service	Acknowledged	Connection	Example
Unacknowledged connectionless	no	no	Ethernet
Acknowledged connectionless	yes	no	802.11 (WiFi)
Acknowledged connection-oriented	yes	yes	long satellite links

- The first is right when errors are rare, and also for voice and video, where late data is worse than bad data.
- The second retransmits a frame that was not acknowledged in time. It is what a wireless link needs and a fibre does not.
- The third numbers every frame and guarantees each one arrives exactly once and in order: a reliable bit stream, built on one hop.

Acknowledging here is an optimisation

- It is never a requirement. The two ends can always send and wait, and repeat the whole thing when nothing comes back.
- The trouble is efficiency: a link has a maximum frame size, so a large packet leaves as ten frames — and if two of them are lost, all ten are sent again.
- The link knows its own frame size, its own delay and its own error rate. The layers above know none of the three.
- So the link can repair cheaply what would otherwise cost a full end-to-end retransmission, seconds later.
- And it removes nothing from above: an optimisation on one hop is not a guarantee over a path.

The check at the bottom of the stack

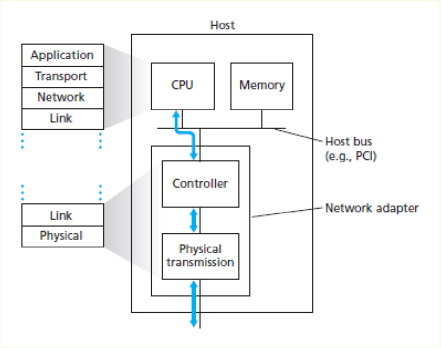
- Bit errors come from attenuation and electromagnetic noise: the receiving hardware reads a one where a zero was sent.
- There is no point in carrying a damaged datagram any further, so the check is made on every hop, by the hardware that received the frame.
- Since nothing further is wrapped around a frame, the check covers the whole of it — headers included, all the way down.
- Being in hardware it can afford to be far more thorough than the sixteen-bit sum the transport layer computes in software.
- Detection or correction: detection says a frame is wrong; correction says which bits are wrong and repairs them. How, is the next lecture's business.

Flow control on one link

- A fast sender and a slow receiver: even with no errors at all, frames arrive faster than they can be taken in, and some are lost.
- Feedback-based: the receiver tells the sender when it may send more — the same shape as the window we saw at the transport layer, over one hop.
- Rate-based: the protocol limits the sending rate with no feedback at all. It belongs to the transport layer, not here.
- Why it has mostly gone away: link hardware is built to run at wire speed: it handles frames as fast as the link can deliver them.
- An overrun is then not the link's fault, and dealing with it is left to the layers above.

Where the link layer lives

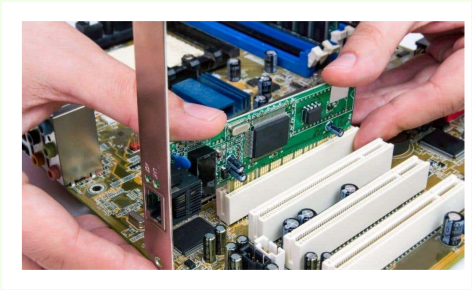
- Most of this layer is not software. It sits on a chip, the network adapter or NIC, with a controller of its own.
- On the way out the controller takes the datagram from host memory, frames it, sets the error-detection bits and transmits.
- On the way in it receives the whole frame, checks it, and hands the datagram up.
- The two halves of the picture meet on the host bus: below it the adapter, above it the CPU and the memory.



What runs on the CPU, what runs on the chip

- Software, on the CPU: addressing, handling the adapter's interrupts, and moving datagrams to and from memory on encapsulation and decapsulation.
- Hardware, on the controller: framing, the error checks, the access rules, and turning the bits into a signal.
- The split is why the layer is fast: what has to be done per bit never touches the processor.
- It is also what a router's input port does at each of its interfaces — terminating the link is exactly this work.
- One more thing lives here: at this layer the interface has a name of its own, the MAC address, unrelated to its IP address. We take it apart with local networks.

From plug-in card to chip on the board



A NIC plugged into a PCI slot

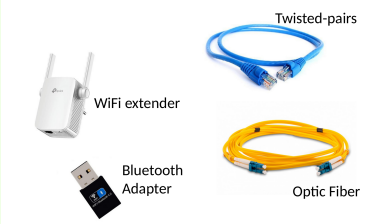


A mini-PC with the NIC embedded

- Adapters used to be separate cards. Now the function sits in the motherboard chipset, or in one cheap dedicated chip.
- What changed is the packaging, not the layering: there is still a controller, and it still does the same work.

The technology decides the frame

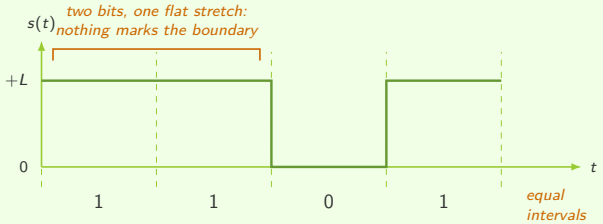
- The link-layer protocol — and therefore the shape of the frame — follows the technology of the physical link.
- Ethernet, 802.3: electrical pulses over four twisted copper pairs, or light over an optical fibre.
- WiFi, 802.11: radio, in the bands around 2.4, 5 and 6 GHz.
- Bluetooth: radio as well, at 2.4 GHz, over short distances.
- So there is no single frame format to learn: there is one per technology, and what they share is the job, not the layout.



What the physical layer hands up

- A raw stream of bits, and nothing else. No marks, no separators, nothing that says where anything starts or stops.
- It is not guaranteed to be correct either. Some bits may have changed value, and the number received may even differ from the number sent.
- Where the channel is noisy the physical layer adds redundancy to its own signals, but that lowers the error rate — it does not remove it.
- Everything above has to be reconstructed from that sequence of zeros and ones, by the layer sitting on top of it.

Nothing in the signal says where a bit ends



- NRZ: one voltage level held for the whole of a bit's interval, with no return to zero in between.
- Two ones in a row give a flat stretch: there is no edge to say where the first bit ended and the second began.
- So the receiver must know by itself where the intervals fall, and the two ends must agree on how long one lasts.
- Let a long run of identical bits go by and the two clocks drift apart: this is why codes force transitions.

Could a link just send an uninterrupted stream of bits?

Two machines, one wire, and a stream of bits that never stops. Would it work?

No. With no boundaries there is nothing an error can be found *in*, and nothing to send again but everything. And the distinct packets handed down by the network layer would arrive as one indistinguishable run.

- Which is why the stream is cut into frames: bounded pieces, each with a beginning, an end, and a check of its own.
- It is the one service no link-layer protocol can do without, and the reason the others become possible.

What framing has to achieve

- The boundaries are not in the bits: they are a structure imposed on a continuous stream, and the receiver has to find them.
- Cheap to find: a receiver must locate the start of a frame easily, and the marks must cost little of the channel.
- Recoverable: after an error the receiver must be able to resynchronise — find where the *next* frame starts, knowing nothing about the one that failed.
- That second requirement is the one that separates the methods, and the one a first attempt always gets wrong.

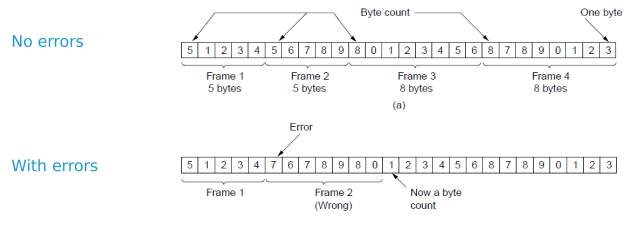
Four ways to find the boundary

Framing methods

1. Byte count.
2. Flag bytes, with byte stuffing.
3. Flag bits, with bit stuffing.
4. Physical layer coding violations.

- The first three work inside the bit stream, and must therefore deal with data that imitates the marks.
- The fourth borrows a signal from the physical layer that the data cannot produce at all.

Byte count



- Each frame begins with a field saying how many bytes follow: the receiver counts them and knows where the frame ends.
- In the lower row one bit flip turned a count of 5 into 7: the receiver takes a data byte for the next count.
- The check says the frame is bad. It does not say where the next one starts, and a retransmission does not help either.
- One flipped bit and the receiver is lost for good: the method is rarely used on its own.

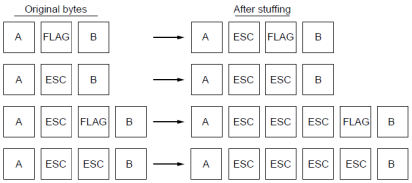
Flag bytes and byte stuffing

Frame format



(a)

Examples



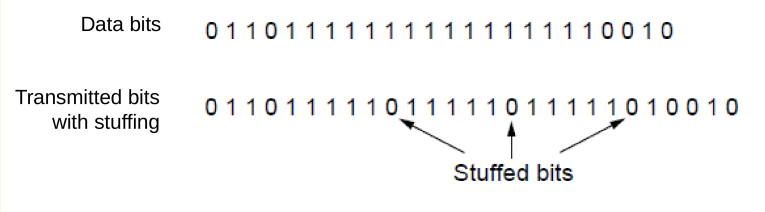
- A FLAG byte marks both ends of a frame. Two flags in a row are the end of one and the start of the next.
- Lose synchronisation and the cure is to scan for two flags: one frame is lost, not the rest of the stream.
- But a flag can occur in the data, and with a photo or a song in the payload it certainly will. Hence byte stuffing: an ESC goes before every FLAG in the data, and before every ESC as well — which is what PPP does.

Destuffing at the receiver



- The rule is mechanical and needs no interpretation: drop the first ESC of a pair and take the byte after it literally.
- What is handed up is exactly what was handed down. The layer above never learns that anything was inserted.
- And the frame boundaries can still be found without undoing the escapes: two FLAGs in a row are two FLAGs either way.

Bit stuffing



- The flag is a bit pattern, 01111110: six ones between two zeros. The frame begins and ends with it.
- After five consecutive ones the sender inserts a zero; the receiver deletes a zero that follows five ones and has the original.
- So six ones in a row can never occur inside a frame: a lost receiver has only to scan for the flag.
- It is not tied to bytes, and it guarantees transitions on the line — which is why USB uses it.

What stuffing costs

What if the payload happens to consist only of flag bytes?

Every one of them is escaped. A hundred bytes of data leave as roughly two hundred: the frame has doubled, and nothing above asked for it.

- With bit stuffing the worst case is milder — one bit added for every five ones, about an eighth — but the property is the same.
- The property: the length of a frame now depends on what is inside it, and neither the sender nor the receiver can predict it from the payload size.
- Which is one reason to want a delimiter that the data cannot imitate at all.

Coding violation

- Physical encodings usually spend more symbols than the data needs, precisely to force the transitions the receiver lives on.
- 4B/5B: four data bits are sent as five signal bits. Sixteen of the thirty-two possible symbols are therefore never produced by data.
- Reserve two of them to mean start of frame and end of frame. They are coding violations: signals the payload cannot imitate, because it has no way to ask for them.
- The gain: no stuffing, nothing grows, and the boundary is unmistakable — the delimiter is outside the alphabet the data can use.
- The price: the method belongs to one physical encoding. Change the encoding underneath and the delimiters are gone.

In practice the methods are combined

- Ethernet and 802.11 begin a frame with a preamble: a fixed pattern, long enough — 72 bits is typical in 802.11 — for the receiver to lock on before anything meaningful arrives.
- In 802.11 and in 802.3 the preamble is followed by a length field, a byte count that finds the end; DIX Ethernet puts a type there.
- So the method that fails on its own is perfectly sound once something else marks the start, and resynchronisation rests on the preamble.
- Which combination a protocol uses is part of its frame format. We take a real one apart, field by field, when we get to Ethernet.

The boundaries are found. And the bits?

The receiver knows where the frame starts and ends. Can it tell whether the bits inside are the ones that were sent?

No. With nothing but the payload there is nothing to compare it against: every sequence of bits is, to the receiver, a perfectly legitimate sequence of bits.

- So the sender computes something redundant over the data and sends it along — the trailer we saw at the start of today.
- The receiver recomputes it and compares. What that something is, how well it works, and when it fails, is where we go next.

Summary

- Below the network layer a datagram travels one hop at a time, inside a frame built and thrown away on that link, by a protocol that may differ at every step.
- Four possible services: framing always; link access where the medium is shared; reliable delivery where errors are frequent; error detection almost everywhere.
- Acknowledging on a link is an optimisation, never a requirement — it repairs locally what would otherwise cost an end-to-end retransmission.
- Most of the layer lives on the adapter's controller, in hardware, because the work is per bit and per frame.
- Framing: byte count, fragile on its own; flag bytes with ESC stuffing; flag bits with a zero after five ones; coding violations the data cannot imitate. In practice, preamble plus length.
- The boundaries are found — and nothing yet says the bits between them are the ones that were sent. That is the next question.